

Networking Question Index - All 60 Questions

Q#	Question
Q1	OSI model - 7 layers, real-world mapping
Q2	TCP/IP model - 4 layers, comparison with OSI
Q3	TCP vs UDP - connection-oriented vs connectionless
Q4	TCP 3-way handshake - SYN, SYN-ACK, ACK, teardown
Q5	IP addressing - IPv4, IPv6, public vs private
Q6	Subnetting and CIDR - calculating subnets, VLSM
Q7	Unicast, Broadcast, Multicast, Anycast
Q8	ICMP - ping internals, traceroute internals
Q9	DNS - resolution, record types, authoritative vs recursive
Q10	DNS caching, TTL, propagation
Q11	HTTP/HTTPS - methods, status codes, headers
Q12	HTTP/2 and HTTP/3 - multiplexing, QUIC
Q13	TLS/SSL handshake - certificates, key negotiation
Q14	Ports and protocols - well-known ports
Q15	MAC addresses, ARP - local device discovery
Q16	DHCP - automatic IP assignment
Q17	NAT and PAT - address translation
Q18	Default gateway and routing basics
Q19	MTU and fragmentation - Path MTU Discovery
Q20	Ping, traceroute, MTR - connectivity testing
Q21	Networking CLI tools - netstat, ss, dig, curl, nc
Q22	tcpdump and Wireshark - packet capture and analysis

Q23	Load balancing - L4 vs L7, algorithms, health checks
Q24	Reverse proxy - Nginx, HAProxy, Traefik
Q25	CDN - edge caching, invalidation, how CDNs work
Q26	VPN - IPSec, WireGuard, OpenVPN, site-to-site
Q27	Firewalls - stateful vs stateless, cloud firewalls
Q28	VLANs - virtual LANs, trunk ports, segmentation
Q29	Proxy servers - forward, SOCKS, transparent
Q30	SSH tunneling - local, remote, dynamic port forwarding
Q31	BGP basics - autonomous systems, peering
Q32	Socket states - ESTABLISHED, TIME_WAIT, CLOSE_WAIT
Q33	Cloud networking - VPC, subnets, SG, peering, TGW
Q34	Kubernetes networking - Pods, Services, CNI
Q35	Docker networking - bridge, host, overlay, macvlan
Q36	Network namespaces - veth pairs, container internals
Q37	Service discovery - DNS-based, consul, CoreDNS
Q38	mTLS and service mesh networking
Q39	WebSockets and gRPC - persistent connections
Q40	nmap - port scanning, network discovery
Q41	Keepalived / VRRP - virtual IP failover
Q42	CORS and API networking
Q43	TCP/IP tuning - buffers, conntrack, backlog
Q44	Network troubleshooting - layer-by-layer methodology
Q45	DDoS mitigation - L3/L4 vs L7 attacks
Q46	Network automation - Ansible, NETCONF, YANG

Q47	App can't connect to database - systematic debugging
Q48	DNS works but HTTPS fails - cert chain, SNI
Q49	Intermittent drops - packet loss, MTU, flapping
Q50	LB health check passes but users get errors
Q51	VPN connected but can't reach internal services
Q52	K8s Pod reaches internet but not another Pod
Q53	tcpdump shows packets but app not receiving
Q54	SSH drops after idle - NAT timeout
Q55	TLS handshake failure - cert chain incomplete
Q56	CLOSE_WAIT accumulating - socket leak
Q57	Subnet overlap - VPC and on-premises conflict
Q58	curl works from one server, fails from another
Q59	Container can't reach host network
Q60	Large packets dropped, small work - MTU mismatch

SECTION 1 - Junior Core Networking (Q1-Q22)

Q1 OSI model - 7 layers

The Simple Answer

The OSI (Open Systems Interconnection) model divides network communication into 7 layers, each handling a specific function. Data flows down the layers on the sending side (encapsulation) and up the layers on the receiving side (decapsulation).

The Analogy - Sending a Letter

Layer 7 (Application) - You write the letter. Layer 6 (Presentation) - You translate it to the recipient's language. Layer 5 (Session) - You establish a conversation ("Dear John... Sincerely"). Layer 4 (Transport) - The post office assigns a tracking number and ensures delivery. Layer 3 (Network) - The postal system determines the route between cities. Layer 2 (Data Link) - The local mail carrier delivers between houses on the same street. Layer 1 (Physical) - The physical truck, road, and mailbox.

Each Layer in DevOps Context

Layer 7 - Application: HTTP, HTTPS, DNS, FTP, SMTP, SSH. This is where your web servers, APIs, and applications operate. WAF inspects traffic here. ALB operates here.

Layer 6 - Presentation: Data formatting, encryption/decryption, compression. TLS/SSL encryption happens here. Character encoding (UTF-8, ASCII).

Layer 5 - Session: Manages connections between applications. Session establishment, maintenance, and teardown. TCP sessions, WebSocket connections.

Layer 4 - Transport: TCP (reliable, ordered) and UDP (fast, connectionless). Port numbers live here. NLB operates here. Firewalls can filter by port.

Layer 3 - Network: IP addresses, routing between networks. Routers operate here. Security groups and NACLs filter by IP. VPCs and subnets are Layer 3 concepts.

Layer 2 - Data Link: MAC addresses, switches, frames. ARP translates IP to MAC. VLANs operate here. Switches forward frames based on MAC addresses.

Layer 1 - Physical: Cables (ethernet, fiber), wireless signals, electrical signals. NICs, hubs, repeaters.

The interview shortcut: "At which layer does a load balancer operate?" ALB = Layer 7 (routes by HTTP path/header). NLB = Layer 4 (routes by IP/port). This is asked in every networking interview.

Q2 TCP/IP model - 4 layers

The Simple Answer

TCP/IP is the practical model that the internet actually uses. It simplifies OSI's 7 layers into 4: Application (OSI 5-7), Transport (OSI 4), Internet (OSI 3), and Network Access (OSI 1-2).

How Data Flows

Application creates data (HTTP request) → Transport adds port numbers and segmentation (TCP header) → Internet adds IP addresses and routing (IP header) → Network Access adds MAC addresses and transmits bits (Ethernet frame). Each layer wraps the previous layer's data with its own header - this is encapsulation. The receiving side unwraps each layer.

Why TCP/IP Matters More Than OSI

OSI is a theoretical reference model - great for understanding concepts. TCP/IP is the actual protocol stack running on every device. When you troubleshoot networking, you think in TCP/IP layers: "Is it a DNS problem (application)? A port issue (transport)? A routing issue (internet)? A cable issue (network access)?"

Q3 TCP vs UDP

The Simple Answer

TCP (Transmission Control Protocol) - Reliable, ordered delivery. Connection-oriented (handshake before data). Guarantees every byte arrives, in order, without corruption. Retransmits lost packets. Used for: HTTP/HTTPS, SSH, FTP, SMTP, database connections - anything where data integrity matters.

UDP (User Datagram Protocol) - Fast, no guarantees. Connectionless (no handshake). Fire and forget - packets may arrive out of order, may be duplicated, or may be lost. No retransmission. Used for: DNS queries, video streaming, gaming, VoIP, monitoring (StatsD), container health checks - where speed matters more than perfection.

The Analogy

TCP is like a phone call - you establish a connection, confirm the other person is listening, and have a reliable conversation. UDP is like shouting across a room - fast, but you are not sure everyone heard you, and you are not repeating yourself.

Key Differences

Feature	TCP	UDP
Connection	3-way handshake	None
Reliability	Guaranteed delivery	Best effort
Ordering	In-order delivery	No ordering
Speed	Slower (overhead)	Faster (no overhead)
Flow control	Yes (sliding window)	No
Use case	Web, SSH, database	DNS, streaming, gaming

DevOps context: Most services use TCP. DNS uses UDP for queries (small, fast) but TCP for zone transfers (large, reliable). Monitoring tools like StatsD use UDP to avoid blocking the application if the monitoring endpoint is slow.

Q4 TCP 3-way handshake

The Simple Answer

Before TCP can send data, client and server must establish a connection through a 3-step process.

Step 1 - SYN: Client sends a SYN (synchronize) packet to the server with a random sequence number. "I want to connect, my starting sequence is X."

Step 2 - SYN-ACK: Server responds with SYN-ACK. "I acknowledge your sequence X, and my starting sequence is Y."

Step 3 - ACK: Client sends ACK. "I acknowledge your sequence Y. Connection established." Both sides now have agreed-upon sequence numbers and can exchange data.

Connection Teardown (4-way)

FIN: One side sends FIN (finished). **ACK:** Other side acknowledges. **FIN:** Other side sends its own FIN. **ACK:** First side acknowledges. The connection is fully closed. The side that initiated the close enters TIME_WAIT state for $2 \times \text{MSL}$ (typically 60 seconds) to handle any delayed packets.

Why DevOps Engineers Care

SYN floods - An attacker sends millions of SYN packets without completing the handshake. The server allocates resources for each half-open connection, exhausting memory. Mitigation: SYN cookies, connection rate limiting, AWS Shield.

TIME_WAIT accumulation - High-traffic servers accumulate thousands of TIME_WAIT sockets after closing connections. Each holds a port for 60 seconds. At high request rates, ports are exhausted. See scenario Q56 and Linux guide Q105.

Q5 IP addressing - IPv4, IPv6, public vs private

IPv4

32-bit address in dotted decimal: 192.168.1.100. Total addresses: ~4.3 billion (exhausted). Each octet is 0-255. Subnet mask defines network vs host portion.

Private IP Ranges (RFC 1918)

10.0.0.0/8 - 16 million addresses. 172.16.0.0/12 - 1 million addresses. 192.168.0.0/16 - 65,536 addresses. Private IPs are not routable on the internet. Used inside organizations. NAT translates private to public for internet access. VPCs use private ranges.

Special Addresses

127.0.0.1 - Loopback (localhost). 0.0.0.0 - All interfaces (listen on all). 169.254.x.x - APIPA (link-local, auto-assigned when DHCP fails). 255.255.255.255 - Broadcast to entire network.

IPv6

128-bit address in hex: 2001:0db8:85a3::8a2e:0370:7334. Vastly more addresses (3.4×10^{38}). No NAT needed - every device gets a globally unique address. Dual-stack (IPv4 + IPv6) is the transition mechanism. AWS VPCs support dual-stack. Kubernetes supports dual-stack networking.

Q6 Subnetting and CIDR

The Simple Answer

Subnetting divides a network into smaller networks. CIDR (Classless Inter-Domain Routing) notation specifies the network size: 10.0.0.0/24 means the first 24 bits are the network, leaving 8 bits for hosts = 256 addresses (254 usable).

Quick Reference

/32 = 1 IP (single host). /24 = 256 IPs (254 usable, most common subnet). /16 = 65,536 IPs. /8 = 16.7 million IPs. Each bit added to the mask halves the network size: /24 has 256 addresses, /25 has 128, /26 has 64, /27 has 32, /28 has 16.

Practical Subnetting for Cloud

AWS VPC CIDR: typically /16 (65,536 IPs). Subnets within the VPC: typically /24 (256 IPs per AZ). Each subnet loses 5 IPs to AWS (network address, VPC router, DNS, future use, broadcast). A /24 subnet gives 251 usable IPs.

The interview question: "You have a /24 subnet. How many usable IPs?" 254 (256 minus network address and broadcast). In AWS: 251 (256 minus 5 reserved). This is asked constantly.

Q7 Unicast, Broadcast, Multicast, Anycast

The Simple Answer

Unicast - One sender to one receiver. Most network traffic is unicast. Your browser talking to a web server.

Broadcast - One sender to ALL devices on the network. ARP requests, DHCP discovery. Limited to the local network (routers do not forward broadcasts). Broadcast storms can crash networks.

Multicast - One sender to a GROUP of receivers who have opted in. Video streaming to multiple viewers. More efficient than unicast (one stream vs one-per-viewer). Used in cluster communication and service discovery.

Anycast - One sender to the NEAREST receiver (among many with the same IP). CDNs and DNS use anycast - your DNS query goes to the nearest DNS server, not a specific one. CloudFront, Google DNS (8.8.8.8), and Cloudflare (1.1.1.1) all use anycast.

Why anycast matters in DevOps: Global services use anycast for low latency. A single IP address (1.1.1.1) resolves to different servers depending on your location. This is how CDNs serve content from the nearest edge location.

Q8 ICMP - ping and traceroute internals

The Simple Answer

ICMP (Internet Control Message Protocol) is a diagnostic protocol. It does not carry application data - it carries error messages and operational information. Ping and traceroute are built on ICMP.

Ping

Sends ICMP Echo Request packets to a target. The target responds with ICMP Echo Reply. Measures: round-trip time (RTT), packet loss percentage. Ping working means Layer 3 (IP) connectivity exists. Ping failing does not necessarily mean the host is down - ICMP may be blocked by a firewall.

Traceroute

Sends packets with incrementing TTL (Time To Live) values. TTL 1: the first router decrements to 0 and sends back ICMP Time Exceeded - revealing the first hop. TTL 2: the second router responds. TTL 3: the third router. This continues until the destination is reached, revealing every hop along the path.

MTR (My Traceroute)

Combines ping and traceroute into a continuous display. Shows packet loss and latency at each hop over time. Essential for identifying intermittent network issues - a single traceroute might miss a problem, but MTR running for 60 seconds catches patterns.

Q9 DNS - resolution, record types

The Simple Answer

DNS translates human-readable domain names (api.example.com) into IP addresses (10.0.1.5). It is a hierarchical, distributed database - no single server holds all records.

Resolution Process

1. Client checks local cache → /etc/hosts. 2. Client queries the configured resolver (ISP or corporate DNS). 3. Resolver checks its cache. If miss, queries root servers. 4. Root servers direct to TLD servers (.com, .org, .io). 5. TLD servers direct to the authoritative nameserver for the domain. 6. Authoritative nameserver returns the IP address. 7. Resolver caches the result (for TTL duration) and returns it to the client.

Record Types

A - Maps domain to IPv4 address. The most common record. **AAAA** - Maps domain to IPv6 address. **CNAME** - Alias pointing to another domain. app.example.com → loadbalancer.aws.com. Cannot be at the zone apex (bare domain). **MX** - Mail exchange server for receiving email. **TXT** - Text records for verification (SPF, DKIM, DMARC, domain ownership). **NS** - Nameserver records - which DNS servers are authoritative. **SOA** - Start of Authority - zone metadata (serial number, refresh interval). **SRV** - Service location records. Used by Kubernetes, LDAP, SIP. Specifies host, port, priority, weight. **PTR** - Reverse DNS - IP to domain mapping.

Authoritative vs Recursive

Authoritative DNS - Holds the definitive records for a domain. "I own example.com and the A record is 10.0.1.5."

Recursive DNS - Queries other servers on behalf of the client, follows the resolution chain, caches results. Your ISP's DNS server is recursive.

Q10 DNS caching, TTL, propagation

The Simple Answer

TTL (Time To Live) - How long a DNS record is cached before a fresh lookup is required. Measured in seconds. TTL 300 = 5 minutes. TTL 86400 = 24 hours. Shorter TTL = faster propagation of changes but more DNS queries. Longer TTL = fewer queries but slower change propagation.

DNS Propagation

When you change a DNS record, the change does not take effect instantly worldwide. Old cached values persist until their TTL expires. With a 24-hour TTL, it can take up to 24 hours for all caches to refresh. Before planned DNS changes, reduce TTL to 60 seconds several hours in advance. Make the change. After propagation, restore the original TTL.

Debugging DNS

dig example.com - Full DNS query with TTL, authoritative server, and flags. dig +short example.com - Just the IP. dig @8.8.8.8 example.com - Query a specific DNS server (bypass local resolver). dig +trace example.com - Follow the full resolution chain from root to authoritative.

The troubleshooting sequence: dig +short (quick answer) → dig +trace (full chain) → dig @specific-server (compare different resolvers). If resolvers disagree, propagation is in progress. Wait for TTL to expire.

Q11 HTTP/HTTPS - methods, status codes, headers

HTTP Methods

GET - Retrieve data. Idempotent. Cacheable. **POST** - Create data. Not idempotent. Sending a form, creating a resource. **PUT** - Replace entire resource. Idempotent. **PATCH** - Partial update. **DELETE** - Remove a resource. **HEAD** - Like GET but returns only headers (no body). Used for health checks. **OPTIONS** - Returns allowed methods. Used in CORS preflight.

Status Codes

2xx Success: 200 OK, 201 Created, 204 No Content. **3xx Redirect:** 301 Moved Permanently, 302 Found (temporary redirect), 304 Not Modified (use cache). **4xx Client Error:** 400 Bad Request, 401 Unauthorized (not authenticated), 403 Forbidden (authenticated but not authorized), 404 Not Found, 429 Too Many Requests (rate limited). **5xx Server Error:** 500 Internal Server Error, 502 Bad Gateway (upstream failed), 503 Service Unavailable (overloaded/maintenance), 504 Gateway Timeout.

Key Headers

Host - Which domain the request is for (allows virtual hosting). **Content-Type** - Data format (application/json, text/html). **Authorization** - Authentication credentials (Bearer token, Basic auth). **Cache-Control** - Caching directives. **X-Request-ID** - Correlation ID for tracing across services.

The DevOps-critical status codes: 502 = your upstream is down (check the backend). 503 = your service is overloaded (scale up). 504 = your upstream is too slow (check timeout settings). These three codes account for most production incidents.

Q12 HTTP/2 and HTTP/3

HTTP/2

Multiplexing - Multiple requests over a single TCP connection simultaneously. HTTP/1.1 sends one request at a time per connection (or opens multiple connections). HTTP/2 interleaves requests and responses on one connection - dramatically faster for pages with many resources.

Header compression (HPACK) - HTTP headers are compressed, reducing overhead on every request. **Server push** - Server proactively sends resources it predicts the client will need. **Binary protocol** - More efficient than HTTP/1.1's text-based format.

HTTP/3

Built on QUIC (Quick UDP Internet Connections) instead of TCP. **0-RTT connection establishment** - First request can be sent with the initial handshake (TCP requires 1-2 RTT before data). **No head-of-line blocking** - If one stream's packet is lost, other streams continue (TCP blocks all streams). **Connection migration** - Switching from WiFi to cellular does not break the connection (TCP connections are tied to IP:port).

Practical impact: Most CDNs and major sites already serve HTTP/2. HTTP/3 is rolling out via Cloudflare, Google, and Meta. Nginx supports HTTP/2 natively. HTTP/3 support requires Nginx with QUIC patches or Caddy server.

Q13 TLS/SSL handshake

The Simple Answer

TLS encrypts HTTP traffic (HTTPS = HTTP + TLS). The handshake establishes a secure connection in 2 round-trips (TLS 1.2) or 1 round-trip (TLS 1.3).

TLS 1.3 Handshake (Simplified)

Client Hello - Client sends: supported TLS versions, cipher suites, and a key share (Diffie-Hellman public value). **Server Hello** - Server responds with: chosen cipher suite, its certificate, and its key share. Both sides now compute the shared secret from the exchanged key shares. **Finished** - Encrypted data flows. All in 1 round-trip (1-RTT). TLS 1.3 is faster than 1.2 because it combines steps.

Certificate Verification

The client checks: Is the certificate signed by a trusted CA? Is the certificate not expired? Does the certificate's Common Name or SAN (Subject Alternative Name) match the hostname? Is the certificate not revoked (CRL or OCSP)?

Cipher Suites

A cipher suite defines: the key exchange algorithm (ECDHE), the authentication algorithm (RSA or ECDSA), the encryption algorithm (AES-256-GCM), and the hash algorithm (SHA-384). TLS 1.3 uses only strong cipher suites - older, weak ones are removed entirely.

Debugging TLS: `openssl s_client -connect host:443` shows the certificate, chain, cipher suite, and TLS version. This is the first command for any HTTPS debugging. See scenario Q55.

Q14 Ports and protocols

The Simple Answer

Ports are 16-bit numbers (0-65535) that identify specific services on a host. An IP address gets you to the machine. The port gets you to the specific application on that machine.

Well-Known Ports for DevOps

Port	Protocol	Service
22	TCP	SSH
53	TCP/UDP	DNS
80	TCP	HTTP
443	TCP	HTTPS
3306	TCP	MySQL
5432	TCP	PostgreSQL
6379	TCP	Redis
8080	TCP	HTTP alternative (Tomcat, Jenkins)
8443	TCP	HTTPS alternative
9090	TCP	Prometheus
3000	TCP	Grafana
9200	TCP	Elasticsearch
2379	TCP	etcd (K8s)

6443	TCP	Kubernetes API
10250	TCP	Kubelet
5601	TCP	Kibana
8500	TCP	Consul
8200	TCP	Vault
27017	TCP	MongoDB

Port Ranges

0-1023: Well-known (privileged). Require root or CAP_NET_BIND_SERVICE. 1024-49151: Registered. Used by applications. 49152-65535: Ephemeral (dynamic). Used for outbound connections. The OS assigns a random ephemeral port for each outgoing connection.

Q15 MAC addresses, ARP

MAC Address

A 48-bit hardware address burned into the network interface: aa:bb:cc:dd:ee:ff. Unique per NIC (in theory). Used for Layer 2 (local network) communication. Switches use MAC addresses to forward frames within a LAN.

ARP (Address Resolution Protocol)

Translates IP addresses to MAC addresses on the local network. When your machine wants to reach 192.168.1.5, it broadcasts: "Who has 192.168.1.5? Tell aa:bb:cc:dd:ee:ff." The device at 192.168.1.5 responds with its MAC address. The result is cached in the ARP table: `arp -a` or `ip neigh show`.

ARP in Containers

Docker and Kubernetes use virtual MAC addresses for containers. The bridge network creates virtual interfaces (veth pairs) with their own MACs. ARP resolution happens within the bridge - the host kernel handles the translation between container MAC and the physical network.

Q16 DHCP

The Simple Answer

DHCP (Dynamic Host Configuration Protocol) automatically assigns IP addresses, subnet masks, gateways, and DNS servers to devices on a network. Without DHCP, every device would need manual IP configuration.

The DORA Process

Discover - Client broadcasts: "I need an IP address." **Offer** - DHCP server responds: "You can use 192.168.1.50." **Request** - Client responds: "I'll take 192.168.1.50." **Acknowledge** - Server confirms: "192.168.1.50 is yours for 24 hours (lease time)."

DHCP in Cloud

Cloud instances get IP addresses via DHCP from the VPC's DHCP service. The DHCP options set configures DNS servers, domain name, and NTP servers for all instances in the VPC. AWS, GCP, and Azure all use DHCP internally for instance networking.

Q17 NAT and PAT

The Simple Answer

NAT (Network Address Translation) - Translates private IP addresses to public IP addresses for internet access. Your internal server at 10.0.1.5 appears as 54.210.100.1 to the internet. NAT is the reason private IPs work - without NAT, private addresses are not routable on the internet.

PAT (Port Address Translation) - Many private IPs share one public IP by using different source ports. Also called NAT overload. Your home router does this - all your devices (phone, laptop, TV) share one public IP, distinguished by different source ports.

NAT in Cloud

AWS NAT Gateway: instances in private subnets access the internet through a NAT Gateway in a public subnet. The NAT Gateway has an Elastic IP (public). All outgoing traffic from private instances appears to come from the NAT Gateway's public IP. GCP Cloud NAT, Azure NAT Gateway serve the same purpose.

Why NAT costs matter in cloud: AWS NAT Gateway charges per hour (\$0.045) AND per GB processed (\$0.045). A busy application transferring 1 TB/month through NAT pays \$45 in processing alone. Use VPC Endpoints for AWS service traffic to bypass NAT. See AWS guide Q127.

Q18 Default gateway and routing

The Simple Answer

A default gateway is the router that handles traffic destined for networks outside the local subnet. When a device wants to reach an IP not on its local network, it sends the packet to the default gateway. The gateway routes it toward the destination.

Routing Table

Every device has a routing table: `ip route show` (Linux). It contains: destination network, gateway, interface, and metric (priority). The most specific match wins - a /32 route overrides a /24 which overrides the default route (0.0.0.0/0).

Cloud Routing

VPC route tables control traffic flow between subnets, internet gateways, NAT gateways, VPN connections, and peering connections. Public subnet: 0.0.0.0/0 → Internet Gateway. Private subnet: 0.0.0.0/0 → NAT Gateway. Peered VPC: 10.1.0.0/16 → VPC Peering Connection.

Q19 MTU and fragmentation

The Simple Answer

MTU (Maximum Transmission Unit) is the largest packet size a network interface can transmit. Standard Ethernet MTU is 1500 bytes. If a packet exceeds the MTU, it must be fragmented (split into smaller packets) or dropped.

Path MTU Discovery

PMTUD discovers the smallest MTU along a path by sending packets with the DF (Don't Fragment) flag set. If a router cannot forward the packet (too large), it sends back an ICMP "Fragmentation Needed" message with the maximum allowed size. The sender reduces the packet size and retries.

Why MTU Causes Problems

VPN tunnels add headers (IPSec adds 50-70 bytes), reducing the effective MTU. VXLAN overlays (used in Docker overlay networks and cloud VPCs) add 50 bytes. If applications send 1500-byte packets through a VPN with 1400 MTU, packets are dropped if DF is set, or fragmented (causing performance issues) if not. See scenario Q60.

The fix: Set MTU on the interface to account for tunnel overhead. For VPN: MTU 1400. For VXLAN: MTU 1450. Or enable PMTUD and ensure ICMP is not blocked (many firewalls block ICMP, breaking PMTUD).

Q20 Ping, traceroute, MTR

Ping

Tests Layer 3 connectivity and measures round-trip time. `ping -c 4 google.com` sends 4 ICMP echo requests. Key output: RTT (round-trip time - min/avg/max), packet loss percentage. Ping working = IP connectivity exists. Ping failing = connectivity issue OR ICMP is blocked.

Traceroute

Shows every hop between source and destination. `traceroute google.com` (Linux) or `tracert google.com` (Windows). Each line is a router along the path. Stars (***) mean the router does not respond to probes - not necessarily a problem. Useful for: finding where packets are being dropped, identifying slow hops, understanding the network path.

MTR (My Traceroute)

Continuous traceroute with ping statistics at each hop. `mtr google.com` runs indefinitely, showing packet loss and latency per hop over time. The best tool for diagnosing intermittent network issues - a single traceroute might look fine, but MTR running for 5 minutes reveals patterns.

Q21 Networking CLI tools

Essential Commands

ss -tlnp - Show listening TCP ports with process names. Replaces netstat. The first command for "what is using this port?"

dig domain - DNS query with full output. `dig +short` for just the answer. `dig @server` for specific resolver. `dig +trace` for full resolution chain.

curl -v URL - HTTP request with verbose output showing headers, TLS handshake, and response. The universal API and web debugging tool.

nc (netcat) - Swiss army knife. `nc -zv host port` tests if a port is open. `nc -l 8080` starts a listener. Used for quick connectivity tests and debugging.

wget - Download files. `wget --spider URL` checks if a URL is accessible without downloading. `wget -r` mirrors a website.

ip addr - Show all network interfaces and IP addresses. `ip route` shows routing table. `ip neigh` shows ARP table.

nslookup domain - Simple DNS query. Less detailed than dig but available on more systems including Windows.

Q22 tcpdump and Wireshark

tcpdump

The command-line packet capture tool. Captures raw packets on a network interface and displays or saves them.

Essential Commands

tcpdump -i eth0 - Capture all traffic on eth0. **tcpdump -i any port 80** - Capture HTTP traffic on all interfaces. **tcpdump -i eth0 host 10.0.1.5** - Capture traffic to/from a specific host. **tcpdump -i eth0 -w capture.pcap** - Save to file for Wireshark analysis. **tcpdump -i eth0 -n port 53** - Capture DNS traffic (-n skips DNS reverse lookup on captured IPs - faster).

Reading tcpdump Output

Each line shows: timestamp, source IP:port > destination IP:port, flags (S=SYN, .=ACK, F=FIN, P=PUSH, R=RESET), sequence numbers, and payload length. SYN packets indicate new connections. RST (reset) packets indicate connection refusal or abort. FIN packets indicate graceful close.

Wireshark

GUI packet analyzer. Opens pcap files from tcpdump. Provides: protocol dissection (breaks down each layer), filters (http.request.method == "POST"), follow TCP stream (reconstruct a complete conversation), and statistics (protocol hierarchy, endpoint statistics).

The debugging power move: "I ran tcpdump on both the client and the server. The client shows the SYN packet being sent, but the server never receives it. This proves the packet is being dropped between them - likely a firewall or security group rule." This level of analysis impresses interviewers.

SECTION 2 - Mid-Level Production Networking (Q23-Q42)

Q23 Load balancing - L4 vs L7, algorithms

The Simple Answer

Load balancers distribute incoming traffic across multiple backend servers for high availability and scalability.

Layer 4 (Transport) - Routes based on IP address and TCP/UDP port. Does not inspect HTTP content. Fastest. AWS NLB, HAProxy TCP mode. Suitable for: any TCP/UDP traffic, databases, non-HTTP protocols.

Layer 7 (Application) - Routes based on HTTP content: URL path, hostname, headers, cookies. Can do SSL termination, path-based routing, header manipulation. AWS ALB, Nginx, HAProxy HTTP mode. Suitable for: web applications, API gateways, microservices routing.

Algorithms

Round-robin - Each server gets the next request in rotation. Simple. Equal distribution assumes equal server capacity. **Least connections** - Send to the server with fewest active connections. Better for uneven request durations. **IP hash** - Hash the client IP to consistently route to the same server. Ensures session affinity without cookies. **Weighted** - Assign weights to servers based on capacity. Server with weight 3 gets 3x more traffic than weight 1.

Health Checks

Load balancers verify backend health before routing traffic. HTTP health checks hit a specific endpoint (/health). If a backend fails the health check, it is removed from the pool. When it recovers, it is added back. Configure: check interval (10 seconds), timeout (5 seconds), unhealthy threshold (3 consecutive failures), healthy threshold (2 consecutive successes).

Q24 Reverse proxy - Nginx, HAProxy, Traefik

Forward vs Reverse Proxy

Forward proxy - Sits in front of clients. Clients send requests to the proxy, which forwards to the internet. Used for: content filtering, caching, privacy (hiding client IP). **Reverse proxy** - Sits in front of servers. Internet traffic hits the proxy, which forwards to backend servers. Used for: load balancing, SSL termination, caching, security (hiding server IPs), compression.

The Big Three

Nginx - The most widely deployed reverse proxy. Handles HTTP, HTTPS, TCP, UDP. Static file serving, SSL termination, caching, rate limiting. Configuration file-based. Excellent performance with low memory footprint.

HAProxy - Specializes in load balancing. Best L4 performance. Advanced health checks, connection draining, stick tables. Configuration file-based. Preferred for high-performance TCP load balancing.

Traefik - Cloud-native reverse proxy. Auto-discovers services from Docker labels, Kubernetes Ingress, and Consul. Automatic Let's Encrypt certificates. Dynamic configuration - no restarts on config change. Preferred for container environments.

Q25 CDN - content delivery networks

The Simple Answer

A CDN caches content at edge locations close to users, reducing latency and offloading traffic from origin servers. User in Tokyo requests an image → the CDN edge in Tokyo serves it from cache (5ms) instead of fetching from the origin in Virginia (200ms).

How It Works

First request: edge has no cache → fetches from origin → caches the response → serves to user. Subsequent requests: edge serves from cache directly. Cache expiry: controlled by Cache-Control headers and CDN TTL settings. Invalidation: force the CDN to refresh content before TTL expires (after deployment).

CDN Providers

AWS CloudFront, Cloudflare, Akamai, Fastly, Google Cloud CDN, Azure CDN. Cloudflare is popular for its free tier and DDoS protection. CloudFront integrates deeply with AWS services.

Q26 VPN - IPSec, WireGuard, OpenVPN

The Simple Answer

VPN creates an encrypted tunnel between two networks or a client and a network. All traffic through the tunnel is encrypted - even on untrusted networks.

IPSec - Industry standard for site-to-site VPNs. Two phases: IKE (key exchange) and ESP (encryption). Complex configuration. Supported by all enterprise routers and cloud providers (AWS Site-to-Site VPN, Azure VPN Gateway).

WireGuard - Modern, simple, fast. Uses state-of-the-art cryptography. Minimal codebase (4,000 lines vs OpenVPN's 100,000+). Integrated into the Linux kernel (5.6+). Excellent for point-to-point connections and road warrior setups. Becoming the default for new deployments.

OpenVPN - Mature, flexible. SSL/TLS-based. Works over TCP or UDP. Extensive configuration options. Large community. Being replaced by WireGuard for new setups but still dominant in existing deployments.

Split Tunnel vs Full Tunnel

Full tunnel - ALL traffic goes through the VPN. Secure but slower (everything goes through the corporate network). **Split tunnel** - Only traffic destined for the corporate network goes through the VPN. Internet traffic goes direct. Faster but less secure (internet traffic is unprotected). See scenario Q51.

Q27 Firewalls - stateful vs stateless

Stateful Firewalls

Track connection state. When you allow outbound traffic, the firewall automatically allows the return traffic (response) - it remembers the connection. AWS Security Groups are stateful - allow inbound port 80, and the response is automatically allowed outbound.

Stateless Firewalls

Evaluate each packet independently. No connection memory. You must explicitly allow BOTH directions - inbound traffic AND the return outbound traffic. AWS NACLs are stateless - you must add rules for both directions, including ephemeral port ranges (1024-65535) for return traffic.

Cloud Firewalls

Security Groups (AWS, GCP) - Stateful, instance-level. Allow rules only (no deny). Evaluated as a whole. **NACLs (AWS)** - Stateless, subnet-level. Allow + deny rules. Evaluated in order. **GCP Firewall Rules** - Stateful, VPC-level. Priority-based. Target by tags or service accounts. **Azure NSGs** - Stateful, subnet or NIC-level. Allow + deny. Priority-based.

Q28 VLANs - virtual LANs

The Simple Answer

VLANs segment a physical network into multiple logical networks at Layer 2. Devices in VLAN 10 cannot communicate with devices in VLAN 20 without a router, even if they are on the same physical switch. This provides isolation, security, and broadcast domain reduction.

Trunk Ports

A trunk port carries traffic for multiple VLANs over a single physical cable using 802.1Q tagging. Each frame is tagged with a VLAN ID so the receiving switch knows which VLAN it belongs to. Trunk ports connect switches to switches, or switches to routers.

VLANs in Cloud Context

Cloud VPCs are essentially VLANs at massive scale. Subnets provide VLAN-like isolation. Security groups and NACLs provide access control. The physical VLAN concepts (tagging, trunking) are abstracted away - you work with logical constructs instead.

Q29 Proxy servers - forward, SOCKS, transparent

The Simple Answer

Forward proxy - Client explicitly configures proxy. Proxy fetches content on behalf of the client. Client's IP is hidden from the server. Use: corporate web filtering, caching, privacy.

SOCKS proxy - Lower-level proxy that works at the TCP level. Handles any protocol (not just HTTP). SSH tunnel with -D flag creates a SOCKS proxy. Used for: bypassing firewalls, routing specific application traffic through a tunnel.

Transparent proxy - Intercepts traffic without client configuration. The client does not know it is being proxied. Uses: corporate monitoring, ISP caching, content filtering. The proxy's IP replaces the client's in outgoing requests.

Q30 SSH tunneling - local, remote, dynamic

Local Port Forwarding

`ssh -L 3306:db-server:3306 bastion` - Access a remote database through a bastion host. Traffic to localhost:3306 is tunneled through SSH to db-server:3306. The database is never exposed to the internet - only accessible through the SSH tunnel.

Remote Port Forwarding

`ssh -R 8080:localhost:3000 server` - Expose your local development server on the remote server. Traffic to server:8080 is tunneled back to your localhost:3000. Used for: showing local work to remote colleagues, webhook testing.

Dynamic Port Forwarding (SOCKS Proxy)

`ssh -D 9050 bastion` - Creates a SOCKS proxy through the bastion. Configure your browser to use localhost:9050 as a SOCKS proxy. All browser traffic is tunneled through the bastion. Used for: accessing internal web UIs through a bastion without individual port forwarding.

The modern alternative: AWS SSM Session Manager port forwarding replaces SSH tunneling for AWS infrastructure. No bastion needed, no port 22, full audit trail. But SSH tunneling remains essential for non-AWS environments and quick debugging.

Q31 BGP basics

The Simple Answer

BGP (Border Gateway Protocol) is the routing protocol of the internet. It determines how traffic flows between autonomous systems (ASes - large networks like ISPs, cloud providers, and enterprises). Every AS has a unique ASN (Autonomous System Number). BGP routers exchange route advertisements - "I can reach 10.0.0.0/16 through my network."

Why DevOps Engineers Care

AWS Direct Connect and VPN use BGP to exchange routes between your network and AWS. Azure ExpressRoute and GCP Interconnect also use BGP. When a BGP misconfiguration occurs, traffic can be routed incorrectly - in 2019, a BGP leak by a small ISP routed traffic through China for 2 hours. AWS Route Origin Authorizations (ROA) protect against BGP hijacking.

Q32 Socket states - ESTABLISHED, TIME_WAIT, CLOSE_WAIT

The Simple Answer

TCP connections go through defined states. Understanding these states is essential for diagnosing connection issues.

LISTEN - Server is waiting for incoming connections on a port. **ESTABLISHED** - Active connection. Data is flowing. **TIME_WAIT** - Connection was closed by this side. Waiting $2 \times \text{MSL}$ (60 seconds) to handle delayed packets. Accumulation exhausts ports at high connection rates. **CLOSE_WAIT** - Remote side closed the connection, but the local application has not closed its end yet. Accumulation indicates an application bug - the app is not closing sockets properly. **FIN_WAIT_1/2** - Connection is closing. Waiting for the remote side to acknowledge or send its FIN. **SYN_SENT** - Connection attempt in progress. If stuck here, the remote is unreachable or firewalled. **SYN_RECV** - Server received SYN but has not completed the handshake. Accumulation indicates SYN flood attack.

Diagnostic Commands

ss -tan shows all TCP connections with states. ss -tan | awk '{print \$1}' | sort | uniq -c | sort -rn shows state distribution. High TIME_WAIT → tune tcp_tw_reuse or use connection pooling. High CLOSE_WAIT → fix the application (it is leaking connections). See scenario Q56.

Q33 Cloud networking - VPC, subnets, SG, peering

The Simple Answer

Cloud networking virtualizes traditional networking concepts. VPC = your private network. Subnets = network segments across AZs. Security Groups = stateful firewalls. NACLs = stateless firewalls. Internet Gateway = internet access for public subnets. NAT Gateway = internet access for private subnets. VPC Peering = connect two VPCs. Transit Gateway = hub connecting many VPCs.

The Standard Architecture

3 public subnets (one per AZ) - load balancers, NAT gateways. 3 private subnets - application servers, containers. 3 isolated subnets - databases (no internet access). VPC Endpoints for AWS services (S3, DynamoDB - free). Route tables per subnet tier.

Q34 Kubernetes networking

The Simple Answer

Kubernetes networking follows a flat model: every Pod gets a unique IP address. Pods can communicate with each other directly without NAT. This is implemented by CNI (Container Network Interface) plugins.

CNI Plugins

Calico - BGP-based. NetworkPolicy support. No overlay (pure L3 routing). High performance. Most popular for on-premises. **Flannel** - Simple VXLAN overlay. No NetworkPolicy support. Easy to set up. Good for small clusters.

Cilium - eBPF-based. Advanced NetworkPolicy, observability (Hubble), encryption. Highest performance. Growing rapidly. **AWS VPC CNI** - Each Pod gets a real VPC IP. No overlay. AWS-native integration. IP address consumption is a concern for large clusters.

Kubernetes Services

ClusterIP - Internal-only virtual IP. Accessible within the cluster. Default service type. **NodePort** - Exposes on every node at a static port (30000-32767). Accessible from outside the cluster. **LoadBalancer** - Provisions a cloud load balancer (ALB/NLB/ELB). The standard for production external access. **ExternalName** - DNS CNAME to an external service.

Q35 Docker networking

Network Modes

bridge (default) - Docker creates a virtual bridge (docker0). Each container gets an IP on the bridge network. Containers on the same bridge can communicate. Port mapping (-p 8080:80) exposes container ports to the host.

host - Container shares the host's network namespace. No network isolation. Container binds directly to host ports. Best performance (no overhead). Security concern (container has full network access).

none - No networking. Container is completely isolated. Used for batch processing that does not need network.

overlay - Multi-host networking for Docker Swarm. VXLAN encapsulation between hosts. Containers on different hosts communicate as if on the same network.

macvlan - Container gets its own MAC address and appears as a physical device on the network. Used when containers need to be directly addressable on the LAN.

Q36 Network namespaces - veth pairs, container internals

The Simple Answer

Network namespaces are the Linux kernel feature that makes container networking possible. Each namespace has its own network interfaces, routing table, iptables rules, and socket list - completely isolated from other namespaces and the host.

How Container Networking Works Internally

1. Docker creates a network namespace for the container. 2. A veth (virtual ethernet) pair is created - two virtual interfaces connected like a pipe. 3. One end (veth-container) is placed in the container namespace. 4. The other end (veth-host) is attached to the docker0 bridge on the host. 5. Traffic from the container flows: container eth0 → veth pair → docker0 bridge → host routing/iptables → destination.

Why This Matters

When container networking breaks, understanding this plumbing is essential. `ip netns list` shows network namespaces. `ip netns exec container_ns ip addr` shows interfaces inside a namespace. `nsenter -t PID -n ip addr` enters a container's network namespace. This is how you debug container networking at the kernel level. See scenario Q59.

Q37 Service discovery - DNS-based, consul, CoreDNS

The Simple Answer

Service discovery answers: "where is the payment service running right now?" In dynamic environments (containers, cloud), IPs change constantly. Hardcoding IPs is impossible. Service discovery provides a stable name that resolves to the current location.

DNS-based (Kubernetes CoreDNS) - Kubernetes creates DNS records for every Service. `payment-service.namespace.svc.cluster.local` resolves to the Service's ClusterIP. Automatic, built-in, no extra tooling. The default for most Kubernetes deployments.

Consul - HashiCorp's service discovery and mesh. Agents register services. Health checks monitor availability. DNS interface for easy querying. Key-value store for configuration. Works across Kubernetes, VMs, and bare metal.

Q38 mTLS and service mesh networking

The Simple Answer

In a service mesh (Istio, Linkerd), a sidecar proxy (Envoy) is injected into every Pod. All traffic between services passes through the sidecar. The mesh provides: automatic mTLS (encrypted, authenticated communication), traffic management (canary, blue-green, circuit breaker), observability (metrics, traces per service pair), and policy enforcement (rate limiting, access control).

How mTLS Works in a Mesh

Service A's sidecar encrypts the request with its certificate → sends to Service B's sidecar → Service B's sidecar verifies A's certificate and decrypts → forwards to Service B's application. Both sides are authenticated and the traffic is encrypted. No application code changes needed - the sidecars handle everything.

Q39 WebSockets and gRPC

WebSockets

HTTP is request-response - client asks, server answers. WebSockets upgrade an HTTP connection to a persistent, bidirectional channel. Both sides can send messages at any time without polling. Used for: real-time chat, live dashboards, collaborative editing, streaming updates.

gRPC

Google's RPC framework. Uses HTTP/2 for transport and Protocol Buffers for serialization. Supports: unary (request-response), server streaming, client streaming, and bidirectional streaming. 10x faster than JSON REST for high-throughput services. Used for: Kubernetes API server, microservice-to-microservice communication, mobile backends.

DevOps Considerations

Load balancers must support WebSocket upgrade (ALB does, some L4 LBs do not). gRPC requires HTTP/2 - some proxies and older load balancers do not support it. Nginx supports gRPC proxying since version 1.13.10. Envoy (Istio sidecar) natively supports both WebSockets and gRPC.

Q40 nmap - port scanning, network discovery

The Simple Answer

nmap is the standard network scanning tool. It discovers hosts, open ports, running services, and OS versions.

Key Commands

nmap -sn 10.0.1.0/24 - Ping scan. Discover live hosts without port scanning. **nmap host** - Scan top 1000 TCP ports. **nmap -p 22,80,443 host** - Scan specific ports. **nmap -p- host** - Scan ALL 65535 ports (slow but thorough). **nmap -sV host** - Detect service versions on open ports. **nmap -O host** - OS detection. **nmap -sU -p 53,161 host** - UDP scan (for DNS, SNMP).

Legitimate Use Cases

Verifying firewall rules - "I configured the security group to allow port 443. Let me verify with nmap." Network inventory - "What services are running on this subnet?" Security auditing - "Are there any unexpected ports open on production servers?" Post-deployment verification - "Is the new service listening on the expected port?"

Warning: Scanning networks you do not own is illegal in many jurisdictions. Cloud providers have specific rules about scanning - AWS allows scanning your own instances without approval, but scanning others' resources is prohibited.

Q41 Keepalived / VRRP - virtual IP failover

The Simple Answer

VRRP (Virtual Router Redundancy Protocol) provides automatic failover for a shared virtual IP address. Two or more servers share a VIP. One is the master (holds the VIP), others are backup. When the master fails, a backup takes over the VIP within seconds.

Keepalived

The standard VRRP implementation on Linux. Used for: load balancer HA (two HAProxy servers sharing a VIP), database failover (two PostgreSQL servers), and any service that needs automatic IP failover without a cloud load balancer.

In Cloud Environments

Cloud providers offer managed load balancers that handle failover automatically. Keepalived/VRRP is less common in cloud but still used for: on-premises HA, bare metal clusters, and hybrid setups where cloud load balancers are not available.

Q42 CORS and API networking

The Simple Answer

CORS (Cross-Origin Resource Sharing) controls which websites can make requests to your API from a browser. Without CORS, a browser enforces the Same-Origin Policy - JavaScript on <https://evil.com> cannot make requests to <https://your-api.com>.

How CORS Works

For cross-origin requests, the browser sends an OPTIONS preflight request. The server responds with headers: Access-Control-Allow-Origin (which origins can access), Access-Control-Allow-Methods (which HTTP methods), Access-Control-Allow-Headers (which custom headers), Access-Control-Allow-Credentials (whether cookies are sent).

Common CORS Mistakes

Setting Access-Control-Allow-Origin: * allows any website to access your API - dangerous if the API uses cookies for authentication. Allowing credentials with a wildcard origin is blocked by browsers (and for good reason). Fix: whitelist specific trusted origins explicitly.

SECTION 3 - Senior Advanced Networking (Q43-Q46)

Q43 TCP/IP tuning - buffers, conntrack, backlog

The Simple Answer

Default kernel network settings are conservative. Production servers handling thousands of connections need tuning.

Key Tuning Parameters

net.core.somaxconn = 65535 - Maximum listen backlog for accepting connections. Default 128 is too low for busy web servers. When exceeded, new connections are dropped.

net.core.rmem_max / wmem_max = 16777216 - Maximum socket receive/send buffer size. Larger buffers improve throughput for high-bandwidth connections. Especially important for long-distance transfers (high bandwidth-delay product).

net.ipv4.tcp_max_syn_backlog = 65535 - Maximum half-open connections waiting for the 3-way handshake to complete. Protects against SYN floods by allowing more pending connections before dropping.

net.netfilter.nf_conntrack_max = 262144 - Connection tracking table size. Every NAT-ed and firewalled connection uses an entry. Exceeding this limit causes "nf_conntrack: table full, dropping packet" - a common production issue for Docker hosts and Kubernetes nodes. Increase for busy hosts.

net.ipv4.tcp_tw_reuse = 1 - Allow reusing TIME_WAIT sockets for new connections. Reduces port exhaustion on busy servers. Safe for client-side connections.

net.ipv4.ip_local_port_range = 1024 65535 - Range of ephemeral ports for outgoing connections. Default range (32768-60999) provides 28,231 ports. Expanding to 1024-65535 provides 64,511 ports - more headroom for TIME_WAIT.

net.ipv4.tcp_keepalive_time = 600 - Seconds before sending keepalive probes on idle connections. Default 7200 (2 hours) is too long - dead connections are not detected for hours. Reduce to 600 (10 minutes).

When to Tune

When you see: "nf_conntrack: table full" → increase conntrack_max. "Cannot assign requested address" → increase port range or enable tw_reuse. Connection refused under load → increase somaxconn and tcp_max_syn_backlog. Slow large file transfers → increase buffer sizes.

Q44 Network troubleshooting - layer-by-layer methodology

The Simple Answer

Systematic troubleshooting works from the bottom up (physical → application) or identifies the likely layer first.

The Methodology

Layer 1 - Physical: Is the cable connected? Is the link up? Check: ip link show (state UP), ethtool eth0 (link detected, speed, duplex).

Layer 2 - Data Link: Is the interface configured? Can you reach the local gateway? Check: ip addr show (IP assigned), arp -a (gateway in ARP table), ping gateway IP.

Layer 3 - Network: Can you reach the remote network? Check: ping remote IP (not hostname - isolate DNS). traceroute to see where packets stop. ip route show to verify routing table. Check firewalls and security groups on both sides.

Layer 4 - Transport: Can you connect to the port? Check: nc -zv host port (TCP connection test), ss -tlnp on the server (is the service listening?). If SYN packets reach the server but no SYN-ACK returns: firewall drop or service not listening.

Layer 5-7 - Application: Is the application responding correctly? Check: curl -v URL (HTTP response, headers, status code). Check application logs. Verify DNS resolution: dig hostname.

The Quick Checklist

Can I ping it? (L3 - IP connectivity). Can I connect to the port? (L4 - nc -zv). Can I resolve the name? (L7 - dig). Does the application respond? (L7 - curl). If all pass but the application fails, it is an application problem, not networking.

The interview answer structure: "I would start by checking Layer 3 connectivity with ping to verify IP reachability. Then Layer 4 with nc or telnet to verify the port is open. Then Layer 7 with curl to verify the application responds. If ping works but the port is closed, I check security groups and the service status. If the port is open but curl fails, I check application logs and TLS configuration. Each layer narrows the problem space."

Q45 DDoS mitigation - L3/L4 vs L7

Layer 3/4 Attacks (Volumetric)

SYN flood - Millions of SYN packets without completing the handshake. Exhausts server resources on half-open connections. Mitigation: SYN cookies (kernel generates the SYN-ACK cryptographically, no state stored until ACK is received), increase tcp_max_syn_backlog, cloud DDoS protection (AWS Shield).

UDP amplification - Attacker sends small requests to public servers (DNS, NTP, memcached) with the victim's IP as the source. The servers send large responses to the victim, amplifying the attack by 50-500x. Mitigation: rate limiting, ingress filtering (BCP38), cloud scrubbing services.

Layer 7 Attacks (Application)

HTTP flood - Millions of legitimate-looking HTTP requests. Cannot be filtered by IP or port - the requests are valid. Mitigation: WAF rules (rate limiting per IP, CAPTCHA challenges, bot detection), CDN absorbs traffic at the edge.

Slowloris - Opens many HTTP connections and sends headers very slowly, holding connections open indefinitely. Exhausts server's connection limit. Mitigation: set timeout values, limit connections per IP, use reverse proxy (Nginx handles this well with worker connections).

Cloud DDoS Protection

AWS Shield Standard - Free. Automatic L3/L4 protection. Always active. **AWS Shield Advanced** - \$3,000/month. Enhanced detection, DDoS Response Team, cost protection. **Cloudflare** - CDN with built-in DDoS protection. Free tier available. Absorbs attacks at edge locations before they reach your origin. **GCP Cloud Armor** - L7 DDoS protection with adaptive ML rules.

Q46 Network automation - Ansible, NETCONF, YANG

The Simple Answer

Managing hundreds of network devices (switches, routers, firewalls) manually is error-prone and slow. Network automation applies DevOps principles to network operations.

Ansible for Network Devices

Ansible connects to network devices via SSH (network_cli) or NETCONF (netconf). Modules like ios_config (Cisco), eos_config (Arista), junos_config (Juniper) manage device configuration. No agent on the device - Ansible runs commands over SSH. Configuration as code - Git-managed, reviewed, tested.

NETCONF / YANG

NETCONF - Protocol for configuring network devices using structured XML. Provides: candidate configurations (make changes, validate, commit or rollback), transactional operations, and formal data models. More reliable than CLI scraping.

YANG - Data modeling language for network configuration and state. Defines the structure of configuration data. Used with NETCONF. Enables: automated validation, programmatic configuration, and vendor-neutral device management.

Why Network Automation Matters

One misconfigured ACL can take down an entire network. Automation provides: version control (Git), code review (PRs), testing (validate before deploying), rollback (revert to previous config), and audit trail (who changed what, when). The same DevOps principles applied to network infrastructure.

SECTION 4 - Tricky Scenarios (Q47-Q60)

Q47 App can't connect to database - systematic debugging

Step-by-Step

Step 1 - Verify the database is running. On the database server: `ss -tlnp | grep 5432` (PostgreSQL). Is it listening? On the expected interface (0.0.0.0 or specific IP)?

Step 2 - Test connectivity from the app server. `nc -zv db-server 5432`. If connection refused: the port is reachable but no service is listening (check Step 1). If timeout: firewall is blocking.

Step 3 - Check firewalls. Security groups: does the app server's SG allow outbound to port 5432? Does the DB's SG allow inbound on 5432 from the app server's SG or IP? NACLs: check both inbound AND outbound (stateless - both directions must be explicitly allowed).

Step 4 - Check routing. Are both hosts in the same VPC/subnet? If different VPCs: is peering established? Are route tables updated? If cross-region: is connectivity set up (peering, TGW, VPN)?

Step 5 - Check DNS. Is the app using a hostname? Does it resolve to the correct IP? `dig db-server` from the app server. If DNS is wrong, the app connects to the wrong host.

Step 6 - Check the database authentication. The connection might succeed at the network level but the database rejects the login. Check DB logs for authentication errors. Verify `pg_hba.conf` (PostgreSQL) allows connections from the app server's IP.

Q48 DNS works but HTTPS fails - cert chain, SNI

Step-by-Step

Step 1 - Test with curl. `curl -v https://api.example.com`. Check the error: certificate verify failed? handshake failure? connection reset?

Step 2 - Check the certificate. `openssl s_client -connect api.example.com:443`. Is the certificate expired? Does the Common Name or SAN match the hostname? Is the issuer trusted? Is the full chain presented (not just the leaf certificate)?

Step 3 - SNI (Server Name Indication). If the server hosts multiple domains on one IP, the client must send the hostname in the TLS ClientHello (SNI). Old clients or misconfigured clients may not send SNI, getting the wrong certificate. `openssl s_client -connect host:443 -servername api.example.com` explicitly sends SNI.

Step 4 - Incomplete certificate chain. The server presents its certificate but not the intermediate CA certificate. Modern browsers can fetch the missing intermediate, but curl and API clients cannot. Fix: configure the web server to send the full chain (server cert + intermediate cert).

Step 5 - Cipher suite mismatch. The client and server have no common cipher suite. Check: `openssl s_client -connect host:443` shows the negotiated cipher. If it fails, there is no match. Fix: update the server's cipher configuration to support modern suites (TLS 1.2+ with AES-GCM).

Q49 Intermittent connection drops

Step-by-Step

Step 1 - Run MTR. `mtr -rw target` for 5 minutes. Identify hops with packet loss. If loss appears at a mid-path hop and continues to the destination, the issue is at that hop. If loss appears at a mid-path hop but the destination shows no loss, the hop is just not responding to ICMP probes (normal).

Step 2 - Check for MTU issues. `ping -M do -s 1472 target` sends a 1500-byte packet with Don't Fragment. If it fails but ping with small packets works, MTU is the issue. Reduce MTU on the interface.

Step 3 - Check for NIC errors. `ethtool -S eth0 | grep error` shows NIC-level errors: RX/TX errors, CRC errors, dropped packets. Hardware issue or cable problem.

Step 4 - Check for conntrack table full. `dmesg | grep conntrack`. If `"nf_conntrack: table full, dropping packet"` appears, increase `net.netfilter.nf_conntrack_max`.

Step 5 - Check for network flapping. `dmesg | grep "link up\|link down"` shows interface flapping. Replace cable, check switch port, check NIC health.

Q50 LB health check passes but users get errors

Why It Happens

The health check endpoint (`/health`) returns 200 OK, but the application endpoints (`/api/orders`) return 500. The health check is too shallow - it checks if the process is running, not if the application is actually working.

Fix

Implement deep health checks. The health endpoint should verify: database connectivity (can I query?), cache connectivity (can I read/write Redis?), downstream service availability (can I reach the payment API?), and application readiness (are background workers running?). If any dependency is unhealthy, the health check returns 503 - the load balancer removes the instance.

Add separate readiness and liveness checks. Readiness: "Can I serve traffic?" (checks dependencies). Liveness: "Am I alive?" (checks process health). Kubernetes uses both. Load balancers should use the readiness check.

Q51 VPN connected but can't reach internal services

Step-by-Step

Step 1 - Check split tunnel vs full tunnel. If split tunnel: only traffic for the corporate CIDR goes through the VPN. Traffic for other IPs goes direct. Is the internal service's IP in the VPN's routing table? `ip route show` should show the corporate CIDR routed through the VPN interface.

Step 2 - Check DNS resolution. Internal DNS names (`service.internal.corp`) may not resolve when connected to VPN if the VPN does not push DNS server settings. Check `/etc/resolv.conf` - is the corporate DNS server listed? If not, configure the VPN to push DNS settings.

Step 3 - Check firewall at the VPN server. Is the VPN server allowing traffic from VPN clients to the internal network? Is there a security group or ACL blocking VPN client IPs?

Step 4 - Check routing at the destination. The internal service must have a route back to the VPN client's IP range. If the VPN assigns clients IPs in `10.8.0.0/24` and the internal server has no route to `10.8.0.0/24`, responses cannot reach the client.

Q52 K8s Pod reaches internet but not another Pod

Step-by-Step

Step 1 - Check if the target Pod is running. `kubectl get pods` - is the target Pod Running? Is the target Service defined correctly? Does the Service selector match the Pod labels?

Step 2 - Check NetworkPolicy. `kubectl get networkpolicy -n namespace`. A default-deny policy blocks all inter-Pod traffic. The source Pod needs an egress policy allowing traffic to the target, AND the target Pod needs an ingress policy allowing traffic from the source.

Step 3 - Check DNS. `kubectl exec source-pod -- nslookup target-service.namespace`. If DNS fails, check CoreDNS Pods (`kubectl get pods -n kube-system | grep coredns`). If DNS works but connection fails, it is a network or policy issue.

Step 4 - Check the CNI plugin. Calico: `calicoctl node status` shows BGP peering status. If nodes are not peering, inter-node Pod traffic fails. Flannel: check that the VXLAN interface is up on all nodes.

Step 5 - Test Pod-to-Pod directly. `kubect exec source-pod -- curl target-pod-ip:port`. If direct IP works but service name does not, the issue is DNS or Service configuration, not networking.

Q53 tcpdump shows packets but app not receiving

What Happened

tcpdump on the server shows incoming TCP SYN packets on port 8080. But the application logs show no incoming connections. Packets arrive at the kernel but never reach the application.

Common Causes

iptables dropping packets. A firewall rule drops the traffic after the network interface but before the application. Check: `iptables -L -n -v` - look for rules with high DROP counts on the relevant port.

Application bound to wrong interface. The app listens on 127.0.0.1:8080 (localhost only) instead of 0.0.0.0:8080 (all interfaces). External traffic cannot reach localhost-bound services. Check: `ss -tlnp | grep 8080` - the Local Address column shows which interface.

SELinux blocking. SELinux may deny the application from binding to the port. Check: `ausearch -m avc` for denial messages.

Socket backlog full. The application's listen queue is full - new connections are silently dropped by the kernel. Check: `ss -tlnp` shows Recv-Q vs Send-Q. If Recv-Q equals the backlog limit, the queue is full. Fix: increase the application's backlog parameter and `net.core.somaxconn`.

Q54 SSH drops after idle - NAT timeout

Why It Happens

A firewall or NAT device between you and the server tracks active connections. After a period of inactivity (often 5-15 minutes), the NAT entry expires and the connection is silently dropped. The next keystroke gets no response - the SSH session appears frozen.

Fix

Client-side keepalive. In `~/.ssh/config`: `ServerAliveInterval 60, ServerAliveCountMax 3`. The client sends a keepalive packet every 60 seconds, keeping the NAT entry alive.

Server-side keepalive. In `/etc/ssh/sshd_config`: `ClientAliveInterval 60, ClientAliveCountMax 3`. The server sends keepalives.

TCP keepalive. Kernel-level: `net.ipv4.tcp_keepalive_time = 300` (send keepalive after 5 minutes of idle). Lower than any firewall timeout in the path.

Use tmux/screen. Even if SSH drops, the session continues. Reconnect and reattach. See Linux guide Q25.

Q55 TLS handshake failure - cert chain incomplete

Step-by-Step

Step 1 - Get details. `openssl s_client -connect host:443`. Check: Verify return code. If it says "unable to verify the first certificate" or "unable to get local issuer certificate" - the chain is incomplete.

Step 2 - Check the chain. The output shows the certificate chain: Certificate chain: 0 s: (your cert), 1 s: (intermediate CA), 2 s: (root CA). If only certificate 0 is present, the intermediate is missing. The server must send the full chain (leaf + intermediate). The root CA is in the client's trust store - it does not need to be sent.

Step 3 - Fix. Concatenate the certificates in order: `cat server.crt intermediate.crt > fullchain.crt`. Configure the web server to use `fullchain.crt`. For Nginx: `ssl_certificate /etc/nginx/ssl/fullchain.crt`. Restart. Verify with `openssl s_client` again.

Step 4 - Verify the fix. `curl -v https://host` should now succeed without certificate errors. Check from multiple clients - some may have cached the old incomplete chain.

Q56 CLOSE_WAIT accumulating - socket leak

What Happened

`ss -tan | grep CLOSE_WAIT | wc -l` shows thousands of `CLOSE_WAIT` connections. The server is running out of file descriptors. New connections fail.

Why It Happens

`CLOSE_WAIT` means the remote side closed the connection (sent FIN), but the local application has not closed its socket. The application received the data but never called `close()` on the socket. This is an application bug - a connection leak. The kernel cannot close the socket - only the application can.

Step-by-Step

Step 1 - Identify the leaking process. `ss -tanp | grep CLOSE_WAIT` shows the PID of the process holding `CLOSE_WAIT` sockets.

Step 2 - Check the application code. Look for: HTTP client connections not closed after use. Database connections not returned to the pool. File descriptors not closed in error paths (try/finally or defer pattern missing).

Step 3 - Temporary relief. Restart the application (clears all sockets). But the leak will recur.

Step 4 - Permanent fix. Fix the code: always close connections in a finally block. Use connection pool libraries that handle cleanup. Set socket timeout so abandoned connections eventually time out. In Java: try-with-resources. In Python: context managers. In Go: `defer conn.Close()`.

Q57 Subnet overlap - VPC and on-premises conflict

What Happened

Your VPC uses `10.0.0.0/16`. Your on-premises network also uses `10.0.0.0/16`. You set up a VPN between them. Routing is confused - traffic for `10.0.1.5` could go to either the VPC or on-premises. Some services are unreachable or reach the wrong server.

Step-by-Step

Step 1 - Confirm the overlap. Check VPC CIDR and on-premises CIDR. If they overlap, direct routing between them is impossible - the kernel cannot determine which `10.0.1.5` you mean.

Step 2 - Short-term fix: NAT. Use NAT to translate between the overlapping ranges. The VPN endpoint translates on-premises `10.0.x.x` to a non-overlapping range (`172.16.x.x`) visible to the VPC. Complex to manage.

Step 3 - Long-term fix: re-address. Change one side's CIDR. In AWS, you can add a secondary CIDR to the VPC and migrate workloads. Plan carefully - IP changes affect: DNS records, security group rules, firewall rules, application configs, and database connection strings.

Prevention

Plan CIDR ranges before building anything. Use unique ranges per environment and location. Document all CIDR assignments centrally. For new VPCs: avoid `10.0.0.0/16` and `172.31.0.0/16` (AWS)

default VPC). Use ranges like 10.100.0.0/16, 10.200.0.0/16 that are unlikely to conflict with on-premises networks.

Q58 curl works from one server, fails from another

Step-by-Step

Step 1 - Compare DNS resolution. dig api.example.com from both servers. Different results? One server may use a different DNS resolver that returns a different IP (or a stale cached IP).

Step 2 - Compare routes. traceroute api.example.com from both servers. Different paths? One server may route through a firewall or proxy that blocks the traffic.

Step 3 - Compare egress rules. Different security groups? One server's outbound rules may block HTTPS (port 443). Check: server A's SG allows outbound 443, server B's SG does not.

Step 4 - Check proxy settings. env | grep -i proxy on both servers. One may have HTTP_PROXY set, routing traffic through a proxy that is blocking or modifying the request. Unset with: unset HTTP_PROXY HTTPS_PROXY.

Step 5 - Check the source IP. The destination may whitelist source IPs. Server A's IP is whitelisted, server B's is not. Common with third-party APIs that require IP whitelisting.

Q59 Container can't reach host network

Step-by-Step

Step 1 - Check the container's network mode. docker inspect container | grep NetworkMode. Bridge mode containers cannot reach the host directly - they must use the host's docker0 bridge IP (typically 172.17.0.1 on Linux). In Kubernetes, use the Node's IP (status.hostIP).

Step 2 - Check iptables. Docker manipulates iptables extensively for NAT and port forwarding. A custom iptables rule may block traffic between the bridge network and the host. Check: iptables -L DOCKER -n.

Step 3 - Check network namespace. nsenter -t CONTAINER_PID -n ip addr shows the container's network interfaces. ip route inside the namespace shows routing. Is the default gateway pointing to the bridge IP? If routing is wrong, the container cannot reach the host.

Step 4 - Docker for Mac/Windows specifics. Docker Desktop runs in a VM. The container's "host" is the VM, not your Mac/Windows machine. Use host.docker.internal to reach the Mac/Windows host from inside a container.

Q60 MTU mismatch - large packets dropped

What Happened

An application works for small requests (API calls returning short JSON) but fails or hangs for large responses (file downloads, large query results). Small packets flow fine. Large packets are silently dropped.

Why

The network path has a link with a smaller MTU than the sender assumes. The sender sends 1500-byte packets. A VPN tunnel, VXLAN overlay, or misconfigured link has an MTU of 1400. Packets exceeding 1400 bytes are dropped. The DF (Don't Fragment) flag prevents fragmentation. ICMP "Fragmentation Needed" messages are blocked by a firewall - the sender never learns about the smaller MTU.

Step-by-Step

Step 1 - Confirm with ping. ping -M do -s 1472 target (1472 + 28 bytes IP/ICMP header = 1500). If this fails but ping -s 1400 target works, MTU is between 1400 and 1500.

Step 2 - Find the exact MTU. Binary search: try 1400, then 1450, then 1425... until you find the maximum packet size that works.

Step 3 - Fix. Set the interface MTU to the discovered value: `ip link set eth0 mtu 1400`. For Docker: configure the docker0 bridge MTU in `/etc/docker/daemon.json`: `{"mtu": 1400}`. For Kubernetes: configure the CNI plugin's MTU setting.

Step 4 - Allow ICMP. Ensure firewalls do not block ICMP Type 3 Code 4 (Fragmentation Needed). This is required for Path MTU Discovery to work. Blocking ICMP is a common cause of mysterious connectivity issues.

The rule: Never block all ICMP in production firewalls. ICMP Type 3 (Destination Unreachable) is essential for network health. Blocking it breaks PMTUD and causes exactly this scenario.

Interview Tips

For Junior Roles

Focus on Q1-Q22. Know OSI/TCP/IP models, TCP vs UDP, subnetting, DNS resolution, HTTP status codes, and basic diagnostic tools (ping, traceroute, curl, dig). Be able to explain how a web request flows from browser to server.

For Mid-Level Roles

Q1-Q42. Be ready to discuss load balancing algorithms, VPN types, container networking (Docker bridge, K8s CNI), and firewalls. Know the difference between L4 and L7. Understand VPC architecture. Have a real story about debugging a network issue.

For Senior Roles

All 60 questions. TCP tuning, DDoS mitigation, network automation, and systematic troubleshooting methodology separate senior from mid-level. Be ready to walk through the layer-by-layer debugging approach for any connectivity scenario.

Quick Reference Cheat Sheet

Topic	Key Takeaway
OSI model	7 layers. ALB = L7. NLB = L4. Switches = L2. Routers = L3.
TCP vs UDP	TCP = reliable, ordered, connection. UDP = fast, fire-and-forget.
Handshake	SYN → SYN-ACK → ACK. TIME_WAIT after close (60 seconds).
Subnetting	/24 = 256 IPs (254 usable). /16 = 65,536. Each bit halves the size.
DNS	dig +trace for resolution chain. A/AAAA/CNAME/MX/TXT records.
HTTP codes	502 = upstream down. 503 = overloaded. 504 = upstream timeout.
TLS	openssl s_client to debug. Full chain = leaf + intermediate.
Ports	22=SSH, 53=DNS, 80=HTTP, 443=HTTPS, 5432=PG, 6379=Redis.
NAT	Private→public translation. Cloud NAT Gateway charges per GB.
MTU	Default 1500. VPN/VXLAN reduce it. Never block ICMP Type 3.
tcpdump	tcpdump -i any port 80 -w capture.pcap for Wireshark analysis.
Load balancing	L4 by IP/port (fast). L7 by HTTP content (smart). Least-conn for uneven workloads.
VPN	WireGuard for new setups. IPSec for enterprise. Split vs full tunnel.
Firewalls	Stateful (SG) = tracks connections. Stateless (NACL) = both directions needed.
Socket states	CLOSE_WAIT = app bug (not closing sockets). TIME_WAIT = normal after close.
Container networking	veth pairs + bridge = Docker bridge. CNI plugin = K8s networking.
SSH tunnel	-L local forward. -R remote forward. -D SOCKS proxy.
BGP	Internet routing. Used by Direct Connect and VPN. Misconfiguration = outage.
Troubleshooting	Ping (L3) → nc port (L4) → dig (DNS) → curl (L7). Layer by layer.
Conntrack	Table full = dropping packets. Increase nf_conntrack_max. Common on Docker hosts.